

Защитная речь — Smart Price (v6, ~9 мин)

Без повторов. SSE-обоснование и кэш встроены в слайд 5 (архитектура).

Каждая мысль звучит **один раз** в самом подходящем месте.

[в курсиве] — можно опустить если поджимает время.

Слайд 1 · Титульный (0:00 — 0:25)

Уважаемая комиссия, здравствуйте.

Меня зовут Санько Егор, группа 4-МД-16. Тема выпускной работы — «**Система метапоиска товаров на торговых площадках с функцией интеллектуального анализа цен**». Научный руководитель — Вагнер Виктория Игоревна.

Слайд 2 · Актуальность · Цель и задачи (0:25 — 1:35)

Чтобы найти лучшую цену в России или Беларуси, покупатель открывает 5–6 вкладок и сравнивает руками. **8–15 минут на сценарий**, при том что динамическое ценообразование даёт разброс в **20–30 процентов в день**. Единого сервиса для обоих рынков на момент начала работы не существовало.

Цель — спроектировать и реализовать такой сервис на базе искусственного интеллекта.

Задач пять: анализ предметной области, проектирование архитектуры, парсеры для шести маркетплейсов, AI-ассистент на Gemini 2.5 Flash, и развёртывание в продакшен.

Слайд 3 · Решение (1:35 — 2:05)

Главная идея — **опрашивать шесть маркетплейсов параллельно** и отдавать каждый результат пользователю сразу, не дожидаясь самого медленного парсера. Параллельно AI-слой сравнивает найденную цену с историей за 60 дней и выносит вердикт «дорого/дёшево».

Слайд 4 · Поиск (2:05 — 2:30)

Реальный пример из прода. Пользователь ввёл просторечное «плойка 5» — AI-коррекция привела к «PlayStation 5», система нашла **38 предложений по шести источникам**.

Слайд 5 · Архитектура (2:30 — 3:50)

Система — это пять слоёв.

Клиент на Next.js подписывается на SSE-поток. Я выбрал именно SSE, а не WebSocket, по одной причине: нужна **только односторонняя передача от сервера к клиенту**. WebSocket — это полнодуплекс, для моей задачи избыточен. SSE проще, нативно поддерживается браузером и проксируется как обычный HTTP.

Edge — nginx с HTTP/2. Критичный момент: nginx по умолчанию буферизует ответ и ломает стриминг, пришлось явно отключить буферизацию на SSE-роутах.

Backend — FastAPI на Python 3.11. Ядро — `asyncio.gather` по шести парсерам и потоковая отдача через `sse_starlette`.

Кэш — Redis в отдельном Docker-контейнере для очередей Celery, плюс in-memory кэш в процессе backend на пять тысяч элементов. Он избавляет от повторных вызовов LLM на одинаковые названия товаров, когда один и тот же артикул приходит с нескольких маркетплейсов.

Всё это — 6 контейнеров на одном VPS-сервере.

Слайд 6 · Парсеры (3:50 — 4:20)

Из 25 изученных маркетплейсов отобрано 6 — по качеству данных и стабильности доступа. У каждого свой подход к парсингу: Onliner отдаёт публичный JSON API, Яндекс Маркет требует headless-Chromium через Playwright, остальные четыре — обычный HTTP-клиент с HTML-парсингом.

Каждый парсер — модуль с единой сигнатурой. Добавление нового источника не требует изменений ядра.

Слайд 7 · AI-слой (4:20 — 4:50)

В качестве LLM выбран Gemini 2.5 Flash через OpenRouter. Три причины: качество русского языка, поддержка стриминга, **стоимость на порядок ниже GPT-4**.

Модель работает по паттерну **tool-use** — сама вызывает функции бэкенда для получения истории цен, сравнения офферов, расчёта медианы. Это исключает выдумывание цифр.

Слайд 8 · «Найти дешевле» — реверс Алисы (4:50 — 5:50)

Уникальная функция работы, **аналогов на рынке нет**.

Голосовой ассистент Яндекса умеет находить «то же дешевле» через закрытый WebSocket-канал. Публичного API нет, поэтому я выполнил **reverse-engineering протокола**: открываю WebSocket с эмуляцией клиента Яндекса, разбираю бинарный поток, извлекаю типизированные офферы `EAliceOfferCard`, `EAliceOffer`, `EAliceOfferList`, и мерджу их в общую ленту по артикулу или EAN.

На реальном кейсе iPhone 17 Pro — найдена цена **119 090 рублей**, это **на 16 113 рублей дешевле, чем на Ozon**. Минус 12 процентов на одной покупке.

[Здесь должна быть гордость в голосе.]

Слайд 9 · Функционал (5:50 — 6:25)

За кулисами — четыре подсистемы.

AI-коррекция запросов — 420 миллисекунд, точность 97 из 100. **Гибридная фильтрация** — regex + AI-классификатор, рост релевантности на 30–40 процентов. **AI-ассистент** с защитой от галлюцинаций — 9 правильных ответов из 10 провокационных тестов. **История цен** за 60 дней по каждому товару.

Слайд 10 · Сравнение и анализ цен (6:25 — 6:50)

Помимо поиска работают ещё две функции: **сравнение** до 4 товаров с AI-обоснованием различий и **анализ цены** — оценка предложения по 100-балльной шкале относительно медианы рынка.

Слайд 11 · Метрики (6:50 — 7:15)

Все числа здесь — **с боевого VPS, не оценочные**. TTFR 1,4 секунды, ускорение в 100 раз по сравнению с ручным поиском, объём кода — 15 тысяч строк.

Слайд 12 · Экономика (7:15 — 7:50)

Сервис в эксплуатации стоит **22 доллара в месяц** — VPS, OpenRouter, домен. При экономии пользователю 2,5–7,5 долларов за один поиск, **3–9 использований окупают месяц работы сервиса**.

[Возможные модели монетизации — контекстная реклама, партнёрские CPA-программы, премиум-подписка.]

Слайд 13 · Инженерные вызовы (7:50 — 8:40)

Четыре основных вызова работы.

Первое — антибот-защиты. Универсального решения нет: для каждого магазина свой подход. **Шесть парсеров стабильно в проде, успешность около 95 процентов.**

Второе — настройка SSE через nginx, о которой я уже говорил.

Третье — реверс WebSocket-протокола Алисы.

Четвёртое — самая свежая правка: поиск «чехол iPhone 16 Pro» возвращал 4 результата вместо 25, потому что фильтры безусловно отсеивали аксессуары. Добавил **детерминистический intent-детектор по 75 regex-паттернам** до обращения к LLM.

Результат: 4 → 25, рост в шесть раз.

Слайд 14 · Заключение (8:40 — 9:00)

Цель работы достигнута. Платформа реализована, работает в продакшене на **smrt-price.ru**, обрабатывает реальные запросы. Все пять задач выполнены, все ключевые решения подтверждены измерениями на проде.

Слайд 15 · Финал (9:00 — 9:10)

Доклад окончен. Благодарю за внимание. Готов ответить на ваши вопросы.

Тайм-чарт

№	Слайд	Время	Длит.
1	Титул	0:00	0:25
2	Актуальность · Цель · Задачи	0:25	1:10
3	Решение	1:35	0:30
4	Поиск	2:05	0:25
5	Архитектура (+SSE +кэш)	2:30	1:20
6	Парсеры	3:50	0:30
7	AI-слой	4:20	0:30
8	Найти дешевле	4:50	1:00
9	Функционал	5:50	0:35
10	Сравнение/Анализ	6:25	0:25
11	Метрики	6:50	0:25
12	Экономика	7:15	0:35
13	Вызовы	7:50	0:50
14	Заключение	8:40	0:20
15	Финал	9:00	0:10
Итого			~9:10

Освободилась **минута запаса** vs v5 (было 10:10).

Что убрано из v5 (повторы)

Что было	Где повторялось	Решение
«параллельный опрос 6 источников»	слайды 3, 5, 11	Сказано один раз на слайде 3, в архитектуре только техника
«SSE-стриминг»	слайды 2, 3, 5, 13	Слайд 5 — техника + обоснование, слайд 13 — только nginx-настройка
«Gemini 2.5 Flash через OpenRouter»	слайды 2, 7	Слайд 7 — единственное упоминание
«25 → 6 маркетплейсов»	слайды 2, 6	Слайд 6
«измерено на боевом VPS»	слайды 8, 11, 14	Слайд 11 — главное упоминание; слайды 8, 14 убраны или сокращены
Полное перечисление парсеров (Onliner, Яндекс, Репард...)	слайды 5, 6	Только слайд 6
«Полный поиск 24,2 секунды»	слайды 3, 11	Только слайд 11
«контекстная реклама / CPA / премиум»	устно вообще убрано из основной речи, осталось в [опциональном]	

NEW SSE + Кэш — встроены в речь на слайде 5

SSE: «Я выбрал SSE, а не WebSocket, по одной причине: нужна только односторонняя передача от сервера к клиенту. WebSocket — полнодуплекс, для моей задачи избыточен. SSE проще, нативно поддерживается браузером и проксируется как обычный HTTP».

Кэш: «Redis в отдельном Docker-контейнере для очередей Celery, плюс in-memory кэш в backend на 5000 элементов — избавляет от повторных вызовов LLM на одинаковые названия товаров».

Это сэкономит **~20 секунд паники на защите**, потому что если спросят — ты уже сам всё проговорил.

Шпаргалка цифр (для вопросов, без изменений)

Метрика	Значение
TTFR (отклик)	1,4 с
Первый батч	5,8 с
Полный поиск	24,2 с
Источников активно	6
Изучено маркетплейсов	25
Ускорение vs ручной	~100x
Стоимость эксплуатации	~\$22/мес
Экономия пользователю	\$2,5–7,5 за поиск
Окупаемость	3–9 использований
AI-коррекция, точность	97 / 100
Рост релевантности фильтрации	+30–40%
Защита от галлюцинаций	9 / 10 тестов
Экономия через Алису	10–40%
iPhone 17 Pro vs Ozon	–16 113 Р (–12%)
Intent-фильтр «чехол iPhone»	4 → 25 результатов
Объём кода	~15 000 строк

Заготовки ответов на вопросы

«Почему SSE, а не WebSocket?»

(Если в речи уже проговорил — короче):

«Как я уже упомянул — нужна односторонняя передача, WebSocket был бы избыточен. Дополню только: у SSE есть встроенный автореконнект, у WebSocket — нет, пришлось бы писать руками».

«А почему не polling?»

«Polling — это лишний трафик и задержки. Клиент дёргает API каждые 500 мс даже когда новых данных нет. SSE — push с сервера, никаких лишних запросов».

«Где хранится кэш?»

(Если в речи уже было — короче):

«Два уровня: Redis для очередей и pubsub-каналов, in-memory словарь в процессе backend для категоризации. Историческая база цен — отдельно в PostgreSQL за 60 дней, это не кэш, а постоянное хранилище».

«Реверс Алисы — это законно?»

«Это исследовательский reverse-engineering в рамках академической работы. Условия использования не нарушены: работаю с трафиком собственного браузерного клиента».

«А если Яндекс изменит протокол?»

«Функция изолирована в отдельный воркер. Если протокол меняется — отключится только она, основной поиск продолжит работать. Предусмотрен мониторинг».

«Почему Gemini, а не GPT или Claude?»

«Gemini Flash через OpenRouter дешевле GPT-4o Mini в 1,5 раза и Claude Haiku в 2,5 раза, при сопоставимом качестве. Плюс OpenRouter решает проблему геоблокировки из России».

«Что с Wildberries и Ozon?»

«Wildberries временно отключён — нестабильные поля наличия. Ozon не подключали — агрессивная защита от ботов, нужна отдельная инфраструктура. Подключение в архитектуре заложено как отдельный модуль».

«Что такое intent-фильтр?»

«Последняя правка перед защитой. Раньше система не различала «iPhone» и «чехол iPhone». Добавил детерминистический детектор по 75 regex-паттернам до обращения к LLM. Закрыло реальный продакшен-баг — поиск чехлов поднялся с 4 до 25 результатов».

«Какая нагрузка на VPS?»

«На пиках до 50% CPU при 5–10 одновременных поисках. Бутылочное горлышко — сами маркетплейсы, их ответ занимает 0,9–1,8 секунды. Памяти — 1,5 ГБ из 4».

«Будет ли мобильное приложение?»

«Архитектура изначально модульная — фронт общается с бэком по REST/SSE. Мобилка на Expo/React Native — отдельная итерация, подключается к тому же API».

Финальные напоминания

- **Главные акценты:** слайд 2 (8–15 минут), слайд 8 (–16 113 ₽), слайд 11 (измерено в проде), слайд 13 (4 → 25).
- **Запас в минуту** — можно медленнее говорить или развернуть один из ответов на вопрос.
- **Если поджимает** — пропускай *[блоки в курсиве]* (модели монетизации, детали парсеров).
- **На «спасибо»** — короткая пауза, кивок, садись.